23CSE455 Design Patterns
Case Study Report

Team:	B14
Case study:	Design Pattern Identification and Evolution in a Next.js-Based Digital News and Publishing Application
Application:	Next.js Blog CMS (faculty approval: [name, date])
Date:	2026-10-10
Submission repository:	23CSE455_DP_CaseStudy

Students

Roll number	Student name	Batch
AM.SC.U4AIE23121	Keziah Suzaine George	AIE
AM.SC.U4AIE23139	Nivedita Ramesh	AIE
AM.SC.U4CSE23360	Soorya Krishna	CSE
AM.SC.U4CSE23309	B R Govind	CSE

1 Application and architecture

Original source. Repository: `https://github.com/thepracticalengineer/nextjs-blog-cms.git`. Baseline commit: `90ec6565877053ee0e93e9f09dcc7c7213aa4832` (merge of pull request #102, 8 October 2026). Access date: 2026-10-10. License: MIT.

Application. A full-stack blog CMS with a TipTap editor, Supabase authentication including MFA, Admin and Author roles, a draft and publication workflow, comments, tags and categories, public author profiles, newsletter subscriptions, an AI writing assistant (Claude, Gemini, OpenAI) and a developer REST API.

Environment.	Node.js	24.14.0 (repository requests 24.21.0; mismatch noted, nothing failed)
	Package manager	pnpm 10.29.3
	Framework	Next.js 16.3.8 (App Router), React 19.3.0, TypeScript 6.0.3
	Database and auth	Supabase (supabase-js 2.117.2, @supabase/ssr 0.12.7)
	Other	Zustand 5.0.15, Zod 4.6.5, Vitest 5.0.3, Playwright 1.63.0

Scope. Inspected: `app/`, `components/`, `features/`, `lib/` and `proxy.ts`. Excluded: `node_modules`, `.next`, `.git`, `.husky`, `.github`, `.agents`, `.claude`, `docs/`, `public/`, `database/`, `supabase/migrations/`, `e2e/`, `__tests__/`, lockfiles and generated files. Reason: not application logic or not relevant to pattern discovery. Unit tests were used only as supporting evidence. The source copy in `code/` equals the baseline commit minus `.husky`, `.github`, `tsconfig.tsbuildinfo` and `package-lock.json`.

Architecture.

The browser reaches public pages, dashboard pages, authentication pages and API routes. Dashboard and authentication paths pass through `proxy.ts` first; its `matcher` covers only `/dashboard`, `/login`, `/register`, `/mfa`, `/forgot-password` and `/reset-password`, so public pages do not. Pages and routes call domain modules in `features/`, which use the Supabase client creators in `lib/supabase`, the permission table in `lib/permissions`, the LLM service and the notification modules.

Request flow (admin opens the Users page). The browser requests `/dashboard/admin/users`. Next.js first runs `proxy.ts`, which refreshes the Supabase session with `supabase.auth.getUser()` (line 34), redirects unauthenticated users to `/login`, sends users with an enrolled but unverified second factor to `/mfa`, and, for paths under `/dashboard/admin`, reads the user's role from the `profiles` table and redirects non-admins to `/dashboard`. When the request passes, the server component `UsersPage` (`app/(dashboard)/dashboard/admin/users/page.tsx`) runs. It

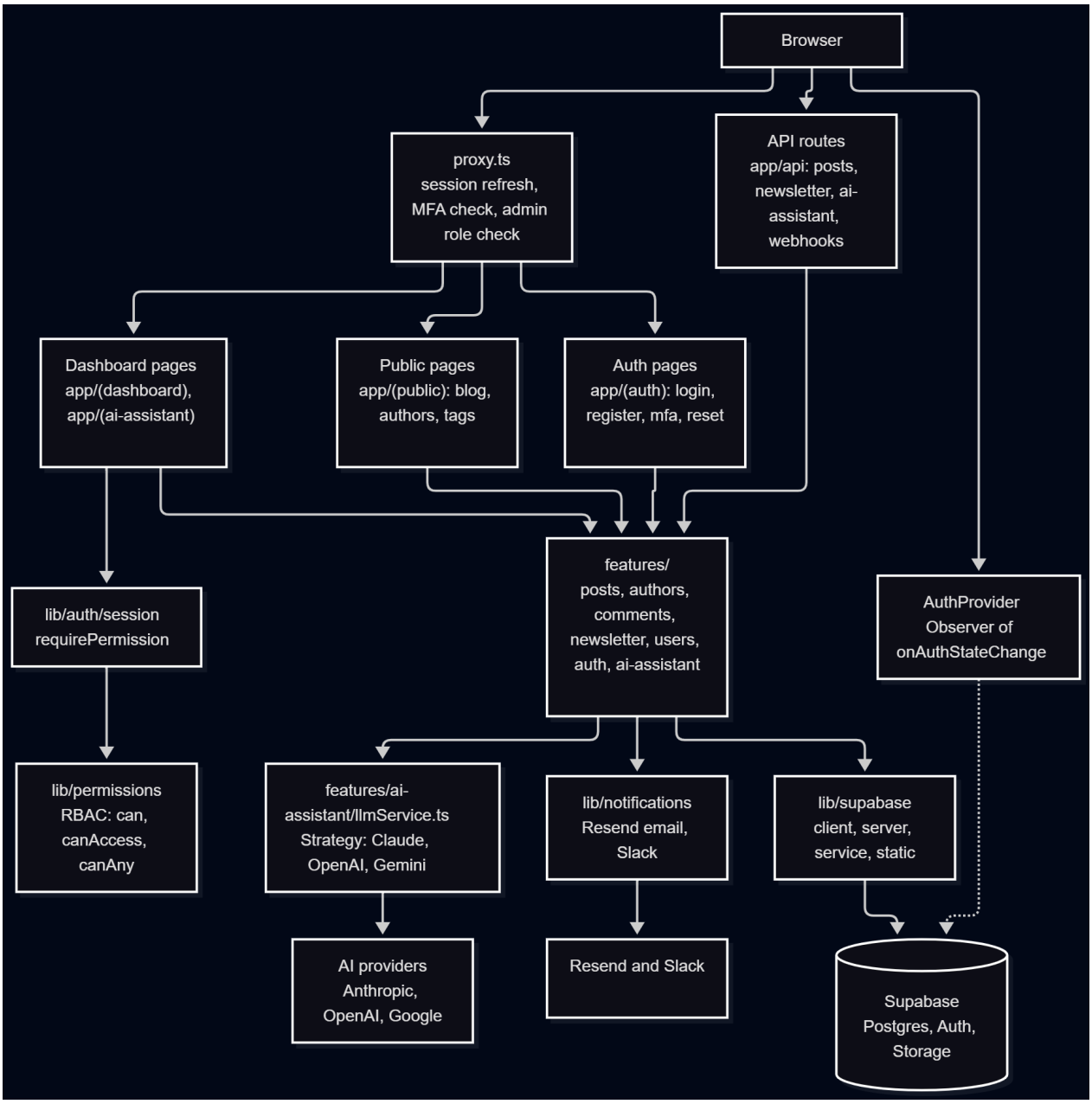


Figure 1: Architecture of the application at the baseline commit.

calls `requirePermission('users:read')` (line 9), loads data with `getAllUsers()` from `features/users/queries` (line 11) and renders `UserTable` (line 21). In the browser, `AuthProvider` keeps session state current by observing `supabase.auth.onAuthStateChange`.

Run instructions. See `README.md`. Results at the baseline:

Command	Result
`pnpminstall--frozen-lockfile`	Succeeded, 933 packages
`pnpmruntest:run`	927 of 927 passed in 83 files on default settings (runs of 71.43 s and 80.10 s). Log: `test_result/test_result_vitest.log`
`pnpmexecvitestrun--testTimeout=` `30000--maxWorkers=2`	83 test files passed, 927 of 927 tests passed (222.58 s). Screenshot: `test_result/vitest_summary.png`
`pnpmruntypecheck`	Passed. Log: `test_result/typecheck.log`; screenshot: `test_result/typecheck.png`
`pnpmrundev`, `pnpmruntest:e2e`	Not run: Docker is not installed, so local Supabase was unavailable

Two further default-setting runs under heavy machine load failed with 6 and 12 failures (130 s and 188 s). Every failure was a 5000 ms timeout in a jsdom UI test, or a follow-on assertion in the same file after a timeout. The repository sits in a OneDrive-synced folder. With a 30 s timeout and 2 workers the same code passed 927 of 927, so the failures were timing-related and not application defects.

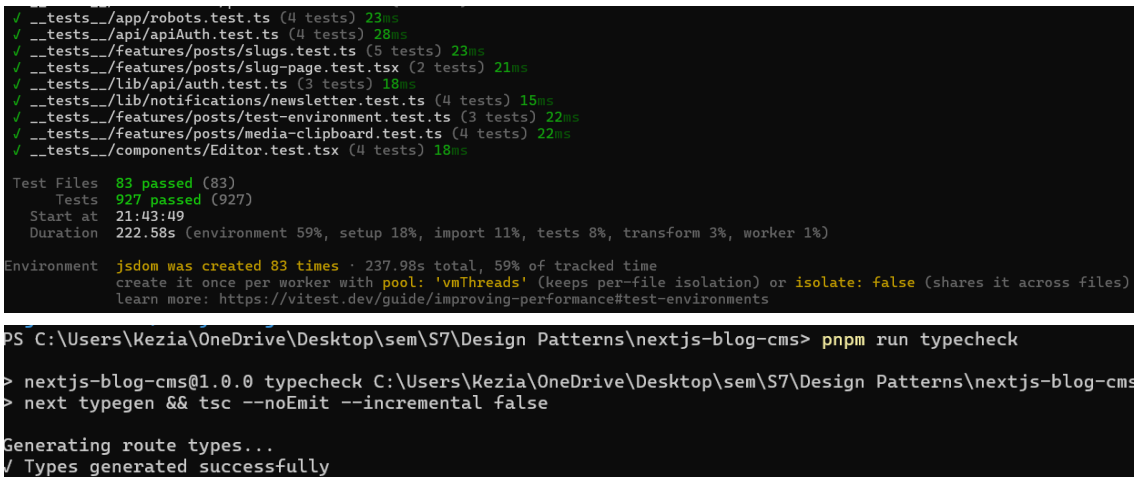


Figure 2: Passing unit-test run and typecheck at the baseline commit.

2 Part 1: Pattern discovery

Method. Candidates were found by reading the file listing and then reading the likely files in full with line numbers (saved in `Part1/evidence/`). Each candidate was checked against three questions: what problem it solves here, whether every role maps to a real symbol, and whether the participants interact as the pattern defines. A class name, a file name or an LLM statement alone was not accepted as evidence. Combined or absent roles are stated in the records; no classes were invented.

2.1 Totals at the baseline

Confirmed pattern types	3 (Strategy, Observer, Role-Based Access Control)
Confirmed pattern instances	3
Concrete participant classes	7, all functions or components rather than classes: `streamClaude`, `streamOpenAI`, `streamGemini`; `AuthProvider`; `can`, `canAccess`, `canAny`
Unique participant symbols	15 application symbols, plus 1 from a dependency (`supabase.auth.onAuthStateChange`)
Rejected candidates	7
Uncertain candidates	1
Observation scope	Files listed in `Part1/rejected_uncertain.md`, at the baseline commit

2.2 Pattern inventory

Pattern type	Instance name	Instances	Concrete classes
Strategy	LLM Provider Streaming Strategy	1	3
Observer	Auth State Observer	1	1
RBAC	RBAC Permission Table	1	3

2.3 Records for each pattern instance

Instance 1: LLM Provider Streaming Strategy

Pattern type and category: Strategy, GoF behavioral. **Purpose and origin:** streams chat replies from Claude, OpenAI or Gemini through one call; application code.

Role	Symbol	File and lines
Strategy	Not separately declared (implicit signature `(params: SendMessageParams)=> AsyncGenerator<string>`)	`features/ai-assistant/llmService.ts`:10–16 (params type)
ConcreteStrategy	`streamClaude`	same file:69–90
ConcreteStrategy	`streamOpenAI`	same file:94–118
ConcreteStrategy	`streamGemini`	same file:129–146
Context	`sendMessage`	same file:150–155

Evidence. The three functions share one parameter type and return type. The context selects one at run time from `params.provider` and delegates:

```
export function sendMessage(params: SendMessageParams): AsyncGenerator<string> {
  if (params.provider === 'claude') return streamClaude(params)
  if (params.provider === 'openai') return streamOpenAI(params)
  if (params.provider === 'gemini') return streamGemini(params)
  throw new Error(`Unsupported provider: ${params.provider}`)
}
```

Limitation. There is no declared interface, and selection is a hard-coded `if` chain rather than injection or a registry.

Verification: Confirmed. Reviewers: team B14. Method: source reading of the listed ranges; `__tests__/ai-assistant/llmService.test.ts` (20 tests) passes in the baseline run but was not

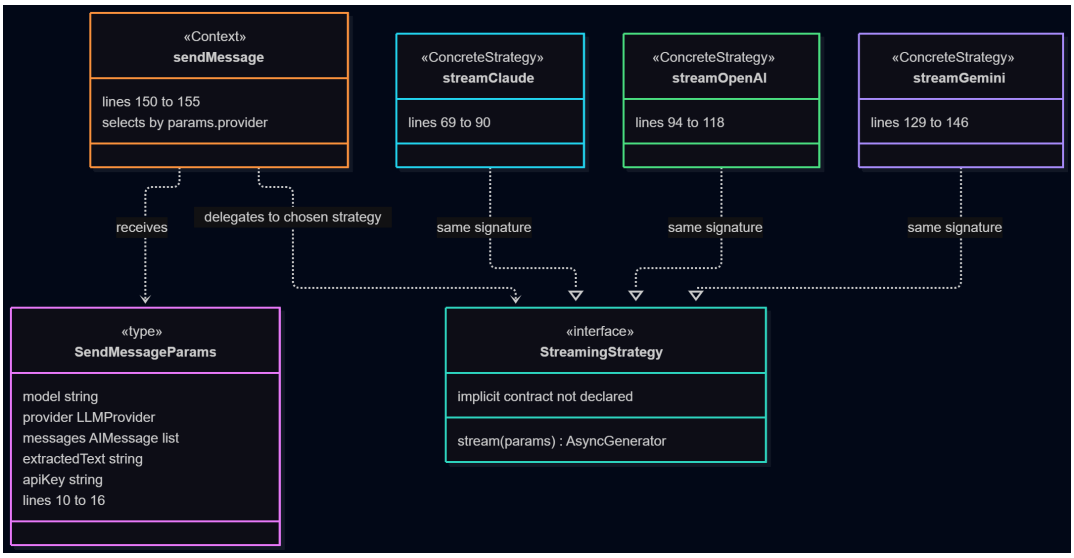


Figure 3: UML for the Strategy instance (source: `Part1/uml_strategy.mmd`).

the sole evidence. Alternative considered: the repeated provider branches in `generateBlogPost`, `generateBlogPostHeadless`, `generateChatTitle` and `validateProviderKey` (lines 157–386) do not apply Strategy; see the rejected list.

Instance 2: Auth State Observer

Pattern type and category: Observer, GoF behavioral. **Purpose and origin:** keeps UI session and profile state in sync with login, logout and token refresh; application code using the Supabase dependency.

Role	Symbol	File and lines
Subject	`supabase.auth` (dependency, not application code)	`features/auth/context/AuthProvider.tsx`:37–39
ConcreteObserver	`AuthProvider`, via the callback given to `onAuthStateChange`	same file:9–59 (callback 39–48)
Detach	`subscription.unsubscribe()`	same file:50
State holder	`AuthContext`, `AuthContext.Provider`	`AuthContext.tsx`:14–19; `AuthProvider.tsx`:54–58
Consumers	`useAuth`; `useSession`	`AuthContext.tsx`:21–23; `features/auth/hooks/useSession.ts`:5–8

Evidence. The component subscribes, updates state when the callback fires, and unsubscribes in the effect cleanup:

```
const { data: { subscription } } = supabase.auth.onAuthStateChange(
  async (_event, session) => { setSession(session); setUser(session?.user ?? null); ... })
return () => subscription.unsubscribe()
```

The subject belongs to `@supabase/supabase-js`, so no subject class is invented. React Context is the delivery mechanism to consumers and is not counted as a second instance.

Verification: Confirmed. Reviewers: team B14. Method: source reading of `AuthProvider.tsx`, `AuthContext.tsx` and `useSession.ts`.

Instance 3: RBAC Permission Table

Pattern type and category: Role-Based Access Control, architecture and security pattern (not a GoF pattern). **Purpose and origin:** centralizes which role may perform which action; application code.

Role	Symbol	File and lines

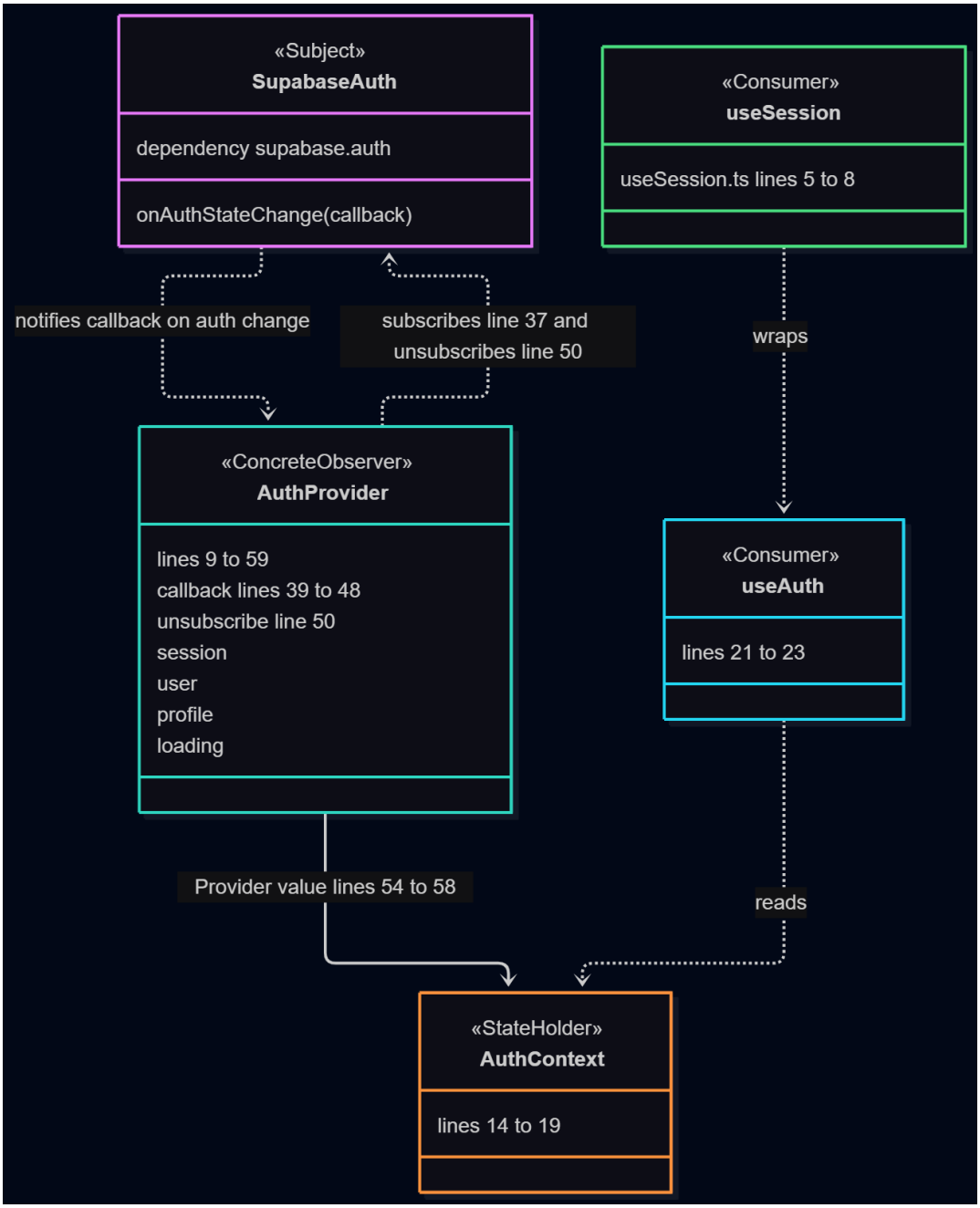


Figure 4: UML for the Observer instance (source: `Part1/uml_observer.mmd`).

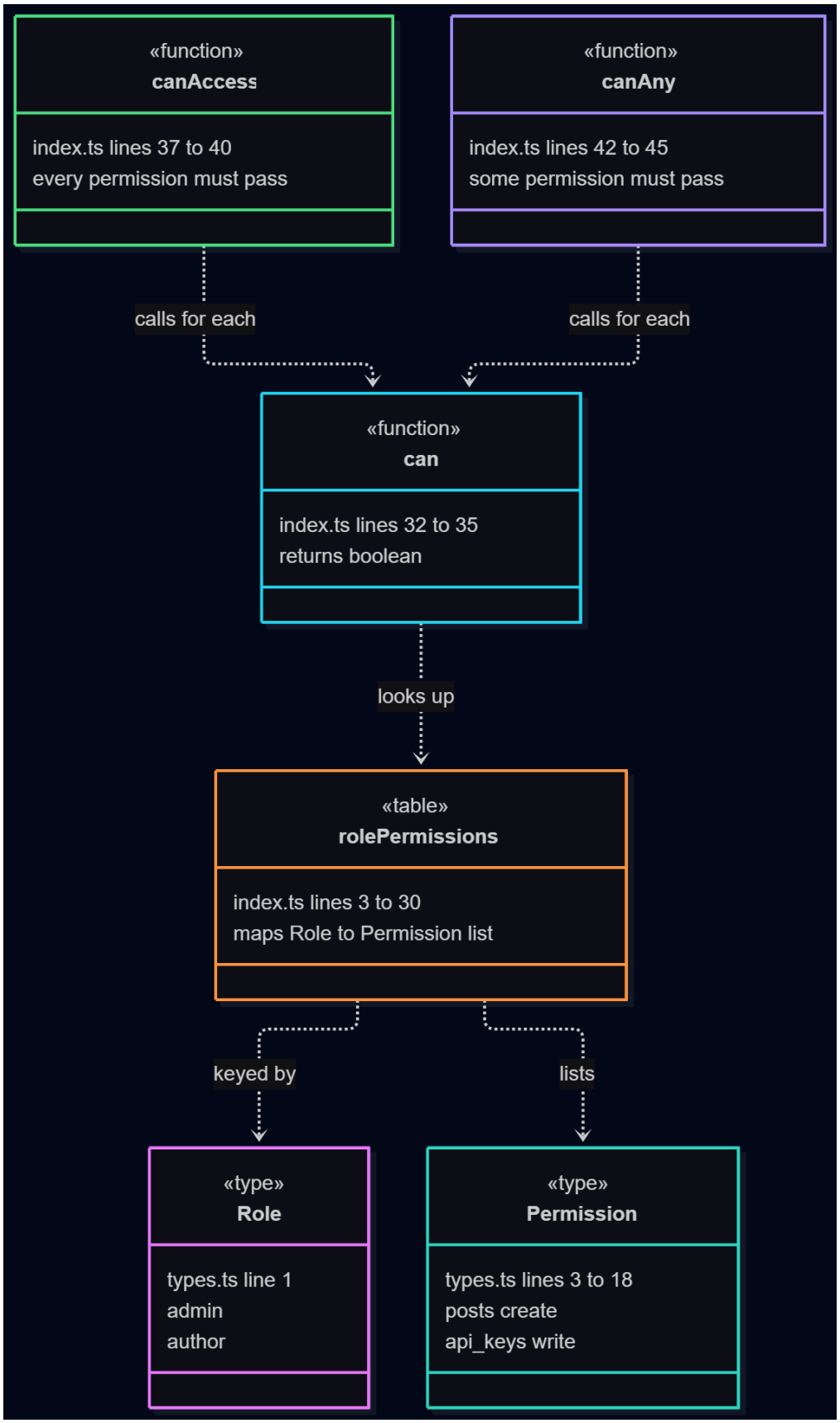


Figure 5: UML for the RBAC instance (source: `Part1/uml_rbac.mmd`).

Verification: Confirmed as an architecture pattern. Reviewers: team B14. Method: source reading; `__tests__/lib/permissions.test.ts` (29 tests) passes in the baseline run.

2.4 Rejected and uncertain candidates

Uncertain (1). `proxy.ts` guard pipeline (lines 4–117): sequential guard checks in one function. It resembles Chain of Responsibility, but there is no handler interface, no successor links and no separate handlers, so roles cannot be mapped without inventing classes. Classified as a Next.js framework idiom.

Rejected (7).

#	Candidate	Reason
R1	Supabase client creators as Factory Method	Four standalone functions chosen by import path; no creator hierarchy or polymorphic selection; `createStaticClient` can return null, so they share no contract
R2	User-confirmed notifications as Observer	`route.ts`:67–74 calls two functions in `Promise.all`, each with its own catch; no subscriber list or notifier interface
R3	`sendNewsletterEmail`	Single function wrapping Resend; one participant
R4	Publication validation as State or Specification	`status` is a plain string; one function with sequential checks, no rule objects
R5	Repeated provider `if` chains	Strategy is not applied; branching duplicated (Part 2 refactoring target)
R6	Helper functions	`buildSystemPrompt`, `resolveGeminiModel`, `isRateLimitError`, `escapeHtml`, `sanitizeSubject`: plain utilities
R7	React Context as a separate Observer	Folded into the Auth State Observer to avoid double counting

3 Part 2: Evolution and refactoring

Three refactoring changes are documented under `Part2/changes/`.

Change	Focus	Implementation and pattern effect	Verification recorded
CH01	AI provider inference	Added `inferProviderFromModel()` and updated `generateBlogPost()` to use it. This centralizes one model-to-provider inference path; the implementation record explicitly says not every provider-selection branch was refactored. Related to the existing Strategy area; do not overstate the change as a complete Strategy redesign.	23 focused tests; 1,025 tests across 88 files; typecheck passed.

CH02	Publication validation	Extracted title checks into `validatePublicationTitle()` and article-content word-count checks into `validatePublicationContent()`. No new GoF pattern is claimed.	30 focused tests; 1,022 tests across 88 files; typecheck passed.
CH03	Generated-post JSON parsing	Extracted common parsing and invalid-JSON handling into `parseGeneratedPostJson()` for the two blog-generation paths. This removes duplication; no new pattern is claimed.	20 focused tests; 1,022 tests across 88 files; typecheck and `gitdiff--check` passed.

Source snapshot note. The uploaded Part 2 folder contains implementation notes and test evidence. However, the copied `code/features/ai-assistant/llmService.ts` in this archive still contains the original provider-selection branches and does not contain the helper names documented for CH01 and CH03. The Part 2 evidence is recorded, but the changed source appears not to have been merged into this copied source tree. This must be reconciled before claiming the archived source itself reflects the post-refactoring implementation.

4 Part 3: Cross-language comparison

This working comparison intentionally covers two confirmed pattern types from Part 1: **Strategy** (LLM Provider Streaming Strategy) and **Observer** (Auth State Observer). RBAC is excluded from this Part 3 comparison to meet the requested two-pattern scope; Part 1 still records three confirmed instances overall.

The Strategy example compares a provider-selection context and concrete provider strategies. The Observer example compares event subscription, notification, and unsubscribe behaviour. Both are implemented in Java and C++ with shared fixtures in `Part3/test-cases/common_test_cases.csv`. These are small deterministic examples of the patterns' intent, not ports of the Next.js application or real calls to LLM providers/Supabase.

Pattern	Java tests	C++ tests
Strategy	4/4 passed	4/4 passed
Observer	4/4 passed	4/4 passed

The dataset `Part3/cross_language.csv` records source LOC, maximum function cyclomatic complexity, toolchain version, test result, and median process time. Five raw timing runs per implementation are saved in `Part3/evidence/timing_raw.csv`; compilation is excluded and process startup/test execution is included. Cyclomatic complexity has been audited function-by-function using the documented McCabe rule `M=1+D`; the decision points and calculations are in `Part3/evidence/complexity_audit.csv` and `Part3/evidence/complexity_audit.md`. This is a transparent manual audit, not output from an external static-analysis package. Timing values are machine-specific and should not be treated as a general language ranking.

Language-scope limitation. The written assignment asks for Java, Python, JavaScript, and C++. This working Part 3 bundle contains Java and C++ only. Python and JavaScript remain outstanding unless the instructor has approved the reduced scope. See `Part3/Part3_Report.pdf` for methodology, results, language notes, and limitations.

5 LLM records and human verification

Run and model. Part 1 pattern analysis used an LLM chat assistant on 2026-10-10. The displayed model name, exact prompts, raw outputs and the evaluation of each analysis are recorded in `Part1/llm.csv`, `Part1/Part1_prompt.txt` and `Part1/llm/`.

Claims and evaluation. Every pattern claim was checked against the numbered source listings in `Part1/evidence/`. Corrections made after verification: the line number given for `getUser()` in `proxy.ts` was wrong and the true line is 34; a claim that the Users page uses `can()` and the server Supabase client was unsupported, since the page calls `requirePermission` and `getAllUsers`, so it was removed; and an initial Observer guess for the user-confirmed notification fan-out was rejected after reading `route.ts`. No generated code was used in Part 1.

6 Findings, contributions and references

Findings. At the baseline commit, three pattern instances were confirmed (Strategy, Observer, RBAC), one candidate is uncertain and seven were rejected. The application mostly relies on framework idioms and plain functions rather than classical GoF structures. The Strategy is function-based with an `if`-chain selector and no declared interface. The Observer subject comes from a dependency. RBAC is an architectural pattern. Of the eight areas in the brief, articles, authors, categories, comments, authentication and newsletter subscription exist; moderation exists only as comment deletion, and recommendations and a paywall were not found. Limitations: the app was not run (no Docker), and behavior was verified from source and the unit-test suite only.

Contributions.

Roll number	Name	Batch	Contribution
AM.SC.U4AIE23121	Keziah Suzaine George	AIE	[actual contribution]
AM.SC.U4AIE23139	Nivedita Ramesh	AIE	[actual contribution]
AM.SC.U4CSE23360	Soorya Krishna	CSE	[actual contribution]
AM.SC.U4CSE23309	B R Govind	CSE	[actual contribution]

References.

1. E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
2. R. Sandhu, E. Coyne, H. Feinstein, C. Youman, "Role-Based Access Control Models," *IEEE Computer* 29(2), 1996.
3. Next.js Blog CMS, `https://github.com/thepracticalengineer/nextjs-blog-cms`, commit 90ec656, MIT license.
4. Next.js documentation, `https://nextjs.org/docs`.
5. Supabase documentation, `https://supabase.com/docs`.
6. Mermaid, `https://mermaid.js.org`.

7 Team folder and datasets

[missing image: test˙result/github˙team˙folder.png]

Figure 6: Team folder B14 in the course repository.

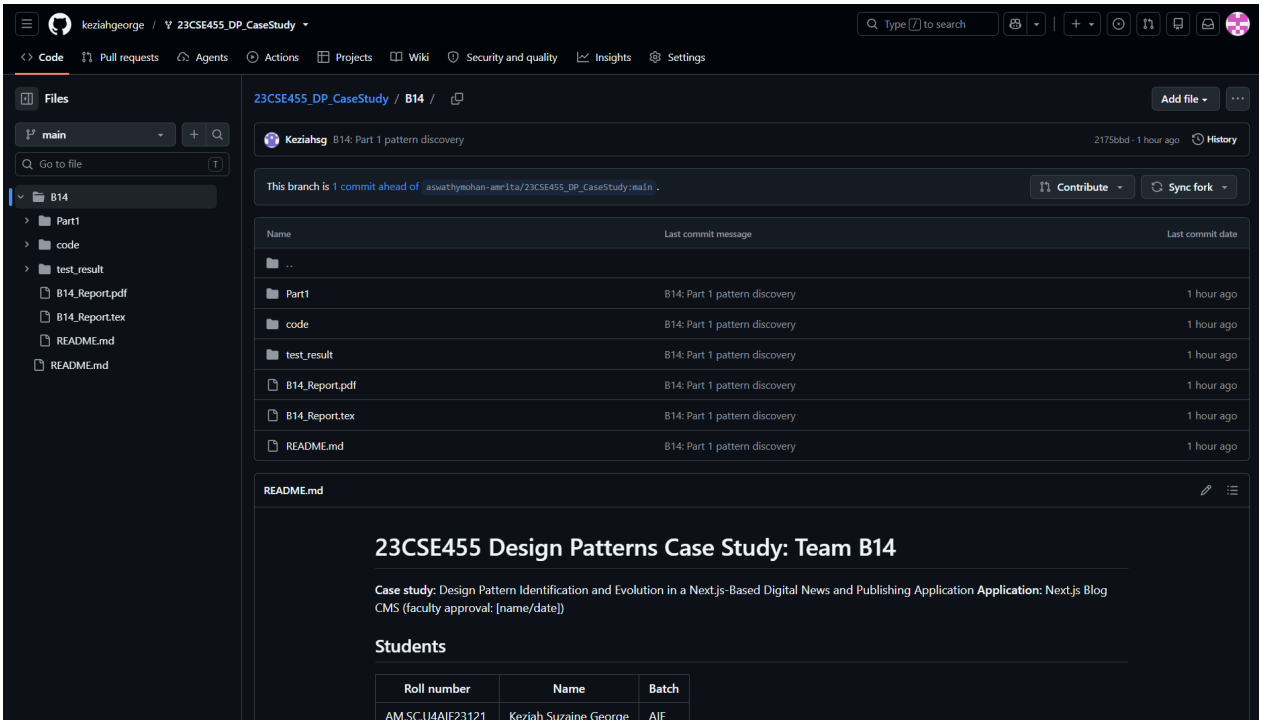


Figure 7: Part1 folder in the course repository.